LLDD BE - Job 2 ImportImpactStore

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

นำเข้าคู่ร้านถูกกระทบจาก ALLMAP: นำคู่ร้านถูกกระทบ–ร้านเปิดใหม่จาก ไฟล์ Impact_Store_Predict_YYYYMMDD.txt บน EAI S3 (interface/in/ALM/ · เปลี่ยนจากวิว SQL Server เมื่อ 2026-09-29) เข้า sgi_fgi_impact_stores เติมข้อมูลจากตาราง master แล้วใช้กฎ DENY และ ON_PROCESS ตั้งค่า verify_status จาก W เป็น P หรือ N

1.2 ขอบเขตงาน

- Main class/script: fgi.main.ImportImpactStore / FGI_ImportImpactStore.sh
- Phase: A
- Output: sgi_fgi_impact_stores
- Estimate: 15 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	20 ชั่วโมง = implementation 15 + unit test 5 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	`ImportImpactStore` (`fcsJar/src/th/co/gosoft/fgi/main/ImportImpactStore.java`)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success,data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-import-impact-store` ใน `src/main.ts` ของ `SBP/srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-import-impact-store INPUT='{"s3Key":"interface/in/ALM/Impact_Store_Predict_20260929.txt"}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"s3Key":"interface/in/ALM/Impact_Store_Predict_20260929.txt"}' sgi-import-impact-store`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: `args[0]` = `ZONES|YYYY|MM` (`ZONES` = `BN`, `BS`, `BW` หรือ `ALL`) · ไม่ส่ง = งวดเดือนก่อนหน้า (`modifyDateToString(now, -1)`)
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่
ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`year`	number	งวดเดือนก่อนหน้า	ค.ศ. 4 หลัก · 2000-2999	`params[1]`
`month`	number	งวดเดือนก่อนหน้า	1-12	`params[2]`
`zones`	string[]	`null` = ทุกโซน	รหัสโซนต้องมีใน `mas_zone` · `[]`/`null`/`"ALL"` = ไม่กรอง	`params[0]` แยกด้วย `,`
`s3Key`	string	`null` = หยิบไฟล์ล่าสุด	key ที่ EventBridge ส่งมา · ต้องอยู่ใต้ `interface/in/ALM/` และขึ้นต้นด้วย `Impact_Store_Predict_`	ของใหม่ — มติ 2026-10-01 (trigger ด้วย S3 event)
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- • มติ 2026-10-01 — ไม่ตั้งเวลาแล้ว · ไฟล์ `Impact_Store_Predict_*.txt` มาถึง `interface/in/ALM/` → EventBridge → `SubmitJob` พร้อม `{"s3Key":"<key>"}` · ทำไฟล์นั้นไฟล์เดียว · ไม่ส่ง `s3Key` = หยิบไฟล์ล่าสุด (รันมือ/rerun)

- ✓ ปิดแล้ว 2026-09-12 — PERIOD_YEAR เป็น ค.ศ. ตรวจฐานเดิมจริง (FGI_IMPACT_STORE_BK_20250515 26,264 แถว) พบปี 2016-2025 เท่านั้น ไม่มีค่า 25xx เลย และคอลัมน์นี้คัดลอกตรงจาก PERIOD_YEAR ของวิว (ImportStoreMapper.java:39) · โค้ดเดิมก็ส่ง ค.ศ. (Locale.US = Gregorian ที่ DateUtils.java:356) ไม่มีการแปลง 543 ที่ใดเลย · ตัวอย่าง ALL|2569|06 ในเอกสารเก่าเป็นตัวอย่างที่เขียนผิด ไม่ใช่พฤติกรรมจริง · INPUT ของ job ใหม่รับ ค.ศ. และยิง ค.ศ. ตรง ๆ (env SGI_JOB2_ALLMAP_YEAR_ERA=AD)

2.2 Input / Output

Stage	Contract for implementation
Input	Period year/month, optional zone filter, and ALLMAP Impact_Store_Predict_*.txt rows read from EAI S3 (interface/in/ALM/).
Progress	query candidate impacted stores, deduplicate by store/month, batch insert impact-store master data, derive related new-store/impact-store records, update verification flags.
Output	FGI_IMPACT_STORE and related impact/new-store tables contain imported candidates for the requested period with duplicate-safe status.

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	อ่านไฟล์ Impact_Store_Predict จาก EAI S3 (กรองงวด/โซน + เลือกแถวแรกต่อคู่ร้านในโค้ด) (interface/in/ALM/ · อ่านไฟล์ตาม s3Key ที่ event ส่งมา (ไม่ส่ง = หยิบไฟล์ล่าสุดตามชื่อที่ลงท้าย YYYYMMDD) · ขาดคอลัมน์ที่ต้องใช้ = ล้มก่อนนำเข้า)
3	มีข้อมูลต้นทาง? | No: จบการทำงาน
4	เป็นคู่ร้านใหม่ (ยังไม่มีใน Oracle)? | No: ข้ามรายการ — ของเดิมไม่ถูกอัปเดต (updateList เป็น dead code) (Errata E4: รันซ้ำจะไม่อัปเดตคู่เดิม)
5	insert คู่ใหม่ verify_status = W (ค่า DEFAULT) (created_by = ALM (คอลัมน์ของ sgi_fgi_impact_stores เอง) · ช่องทางต้นทางระดับรอบชดเชยอยู่ที่ sgi_fgi_impact_processes.datasource)
6	เติมข้อมูล master และ enrichment data (INNER JOIN — ถ้า master ไม่ครบ แถวจะหลุดหายเงียบ ๆ)
7	ผ่านกฎ DENY? (ตรวจก่อน ON_PROCESS) | No: verify_status = N (Deny) (DENY: สาขา N=F / juristic เดียวกัน / สัญญา SBP ไม่คลุมงวด / เก่ากว่า 12 เดือน)
8	เข้าเงื่อนไข ON_PROCESS หรือ sgi_fgi_impact_stores.created_by = STA? | No: คงค่า verify_status = W (รอตรวจสอบ) (แถวที่ created_by = STA เขาสถานะ P ได้อัตโนมัติ)
9	verify_status = P (On Process) แล้ววนจนครบทุกแถว (sales_request_status ไม่ถูกแตะเลย — คงเป็น W ตาม DEFAULT ให้ Job 4 หยิบต่อ)
10	จบ

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 2 ตัดสิน 3 เรื่องต่อกันเป็นทอด และ ผิดข้อใดข้อหนึ่งก็ทำให้ร้านหายไปเงียบ ๆ โดยไม่มี error: (ก) แถวไหนจาก ALLMAP นับเป็น candidate (ข) candidate นั้น เป็นคู่ร้านใหม่หรือมีอยู่แล้ว (ค) คู่ที่ insert แล้วจะ เข้ากระบวนการ หรือ ถูกตัดทิ้ง

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
ก. แถวนี้เป็น candidate ของงวดหรือไม่	ไฟล์ `Impact_Store_Predict_YYYYMMDD.txt` บน EAI S3 (`interface/in/ALM/`) — ● เปลี่ยนต้นทาง 2026-09-29 จากวิว `allmapssa.SEVEN_IMPACT_VIEW` (SQL Server) มาเป็นไฟล์ · pipe-delimited · UTF-8 · LF · มีหัวตาราง · 21 คอลัมน์ (วัดจากไฟล์จริง 546 แถว)	กรองงวดในโค้ด: `PERIOD_YEAR` = ปีที่ขอ และ `PERIOD_MONTH` = เดือนที่ขอ · ● `PERIOD_MONTH` ของไฟล์นี้เป็นวันที่เต็ม `20260801` ไม่ใช่เลขเดือน จึง normalize ทั้งสองฝั่งก่อนเทียบ (ไฟล์คู่แข่งของ Job 3 เป็นเลขเดือน 8) · ส่ง `zones` มาก็กรอง `ZONE_I` ในโค้ด · เก็บเฉพาะแถวแรกของแต่ละ `(STORECODE_I, STORECODE_N, PERIOD_YEAR, PERIOD_MONTH)` เรียงด้วย `DISTANCE ASC, STORECODE_N ASC` (แทน `ROW_NUMBER() OVER(...)` ของวิว · ค่าที่แปลงเป็นเลขไม่ได้ไปท้ายสุด ไม่ใช่กลายเป็น 0)	ไม่เข้าเกณฑ์ = ไม่อ่านเข้ามา · ไฟล์ไม่มีแถวของงวดนี้ = จบงานแบบ SUCCESS พร้อม note IF01002 (ระบบเดิม `return true` ไม่ใช่ FAIL) · อ่านไฟล์ไม่สำเร็จ/ขาดคอลัมน์ = `ALLMAP_READ_FAILED` (ล้มก่อนนำเข้าบางส่วน)
ข. เป็นคู่ร้านใหม่หรือมีอยู่แล้ว	`sgi_fgi_impact_stores` (`impacted_store_code` · `new_store_code` · `impact_month` · `verify_status`) LEFT JOIN `sgi_fgi_impact_processes` (`impacted_store_code` · `flag_action` · `start_compensate_year/_month` · `end_compensate_year/_month`)	สร้าง ชุด "มีอยู่แล้ว" ของงวดก่อน (SQL ด้านล่าง) แล้วเทียบ candidate ด้วยคีย์ `(impacted_store_code, new_store_code)` · ไม่พบในชุด = คู่ใหม่ → INSERT · พบ = ข้าม	▲ พบแล้วระบบเดิมไม่อัปเดตอะไรเลย — `updateList` ถูกสร้างขึ้นแต่ `manageImpactStore()` เรียกเฉพาะ `insertList` (`ImpactStoreService` บรรทัด 45-51, 148-155) · ระบบใหม่ต้องคงพฤติกรรมนี้และนับเป็น `skipped`
ค1. คู่นี้ต้อง ถูกตัดทิ้ง หรือไม่ (`verify_status = 'N'`)	ประเภทสาขา = `sgi_fgi_impact_stores.branchtype_i` / `branchtype_n` (ค่าที่ ALLMAP ส่งมา · มติ 2026-10-05) — แถว STA และแถวที่ยังไม่มี snapshot ใช้ `mas_store.status_type` แทน · ตารางของระบบ SBP เดิม (อ่านอย่างเดียว): `mas_store` (`branch_id` · `status_type` · `open_date` · `region`) · `fr_store` (`store_id` · `juristic_id` · `start_date` · `cancel_date` · `cancel_type` · `status` · `order_id`) · `juristic` (`juristic_id` · `juristic_name`)	ตัดทิ้งเมื่อ ข้อใดข้อหนึ่ง จริง: ประเภทสาขาฝั่ง I ไม่อยู่ในรายการที่รับได้ · ฝั่ง N เป็น F · นิติบุคคลสองฝั่งเป็นรายเดียวกัน · สัญญา SBP ของร้าน I ไม่คลุมงวด · หรือแถวเก่ากว่า 12 เดือน (SQL ด้านล่าง)	ตั้ง `verify_status = 'N'` + `updated_at` และ ไม่ลบแถว (เก็บไว้ตรวจย้อนหลัง)
ค2. คู่นี้ เข้ากระบวนการ หรือไม่ (`verify_status = 'P'`)	ชุดคอลัมน์เดียวกับ ค1 + `sgi_fgi_impact_stores.created_by` / `.updated_by`	เข้ากระบวนการเมื่อ มาจาก ALLMAP และ ผ่านเกณฑ์ประเภทสาขาทั้งสองฝั่ง และ นิติบุคคลคนละราย และ สัญญา SBP คลุมงวด — หรือ มาจาก STA (เคสที่ระบบ Statement ส่งเข้ามาเอง ผ่านทันทีโดยไม่ตรวจเกณฑ์)	ตั้ง `verify_status = 'P'` + `updated_at` · แถวที่ไม่เข้าทั้ง ค1 และ ค2 ค้างเป็น `'W'` ให้รอบถัดไปหยิบ

ช่องว่างที่เคยค้างของหัวข้อนี้ — ปิดครบแล้ว (เก็บไว้เป็นประวัติ)

ทุกข้อปิดแล้ว — เก็บตารางไว้เพื่อให้ตามรอยได้ว่าเคยค้างอะไรและปิดด้วยอะไร

#	สิ่งที่ขาด	ต้องทำอะไรก่อน
G1 ✓ ปิดแล้ว 2026-09-02	เดิม `sales_request_status` (`W/P/Y/E`) ไม่มีที่เก็บผล DENY	เพิ่มคอลัมน์ `verify_status CHAR(1) CHECK IN ('W','P','N')` แยกจาก `sales_request_status` — legacy มีสองสถานะคนละเรื่อง (ตรวจคู่ร้าน vs ขอยอดขาย) ที่โครงเดิมยุบเหลือคอลัมน์เดียว

#	สิ่งที่ขาด	ต้องทำอะไรก่อน
G2 ✓ ปิดแล้ว 2026-09-02	เดิมไม่มี `created_by` / `updated_by`	เพิ่ม `created_by` / `updated_by` `VARCHAR(10) CHECK IN ('ALM','STA','USER')` และ `created_at` (กฎ "เก่ากว่า 12 เดือน" อ้างคอลัมน์นี้)
G3 ✓ ปิดแล้ว 2026-09-12 · ● ปรับ 2026-10-05	ไม่มีคอลัมน์วันเปิดร้าน / ประเภทสาขา / นิติบุคคล / วันสัญญา SBP ใน `sgi_*` เลย	ประเภทสาขา — มติผู้ใช้ 2026-10-05 (ทาง ก.) เก็บ snapshot ตามระบบเดิม: เพิ่ม `sgi_fgi_impact_stores.branchtype_i` / `branchtype_n` เก็บค่าที่ ALLMAP ส่งมา แล้วตัดสินแบบ `decode(create_by,'STA', master, snapshot)` ของเดิม (แถวที่นำเข้าก่อนมีคอลัมน์ถอยไปใช้ master) · ข้อมูลเดิมที่เคยวัดไว้ว่าเปลี่ยนไปใช้ master แล้วผลพลิก 19 แถว (0.08%) ไม่ได้ทำให้มตินี้เปลี่ยน เพราะเป้าหมายคือผลตรงระบบเดิมและตรวจย้อนหลังได้ · นิติบุคคล / วันสัญญา SBP ยัง join `fr_store` · `juristic` ของ schema `sps_store` (อ่านอย่างเดียว) เหมือนระบบเดิม · สิ่งที่เหลือคือ ยืนยันสิทธิ์อ่านข้าม schema + index บน `mas_store.branch_id` / `fr_store.store_id` (ข้อมูลจริง: ร้าน 1,056 ร้านมีสัญญา `fr_store` มากกว่า 1 ฉบับ — ชั้น `dense_rank() = 1` จำเป็นจริง ไม่ใช่ทฤษฎี)

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
`verify_status` (เดิม `FLAG_VERIFY`)	`W` = รอตรวจ (ค่าตั้งต้นตอน insert) · `P` = เข้ากระบวนการ · `N` = ถูกตัดทิ้ ข้อมูลจริง (26,264 แถว): `N` 14,424 · `Y` 8,556 · `W` 3,064 · `P` 220 — `Y` ของเดิมคือ "ยอดขายผ่านแล้ว" ซึ่งโครงใหม่แยกไป `sales_request_status` migration ต้องแมป ไม่ใช่คัดลอกตรง	`FgiConstant.FLAG_VERIFY_WAIT` / `_ON_PROCESS` / `_DENY` · ตรวจฐาน stqa 2026-09-12
`created_by` / `updated_by` (เดิม `CREATE_BY`/`UPDATE_BY`)	`ALM` = ALLMAP · `STA` = ระบบ Statement ส่งเข้ามา · `USER` = คนคีย์เอง ข้อมูลจริง: `CREATE_BY` = ALM 23,126 · STA 3,138 (ไม่มี USER เลย) · `UPDATE_BY` = NULL ทั้ง 26,264 แถว → เงื่อนไข `'ALM' IN (created_by, updated_by)` ของกฎตัดทิ้ง ในทางปฏิบัติเทากับ `created_by = 'ALM'`	`FgiConstant.ALLMAP` / `FRANCHISE_STATEMENT` · ตรวจฐาน stqa 2026-09-12
ประเภทสาขาฝั่ง I ที่รับได้	`B` · `FAM` · `FB1` · `FB2` · `FC1` · `FVB` · `FVC` — และ `FPT1` เฉพาะเมื่อ ประเภทการยกเลิก SBP = `'06'`	SQL ของ `updateImpactStoreByJuristicMeetCondition()`
ประเภทสาขาฝั่ง N ที่ห้าม	`F`	เงื่อนไข `a.branchtype_n in ('F')` ในกฎตัดทิ้ง
หน่วยของ `DISTANCE_UNIT` ที่ ALLMAP ส่งมาจริง	กิโลเมตร (ภาษาไทย) ค่าเดียวเท่านั้น — 23,127 แถวของ `CREATE_BY = 'ALM'` ทั้งหมด · ไม่เคยส่ง `KM`/`M` ภาษาอังกฤษเลย · แถว STA ส่ง `DISTANCE = 0` และหน่วยเป็น NULL ระยะทางจริงอยู่ในช่วง 0.03–1.52 กม. และคอลัมน์เดิมเป็น `NUMBER(3,2)` = เก็บเป็นกิโลเมตรอยู่แล้ว	ตรวจฐาน stqa 2026-09-12 — whitelist การแปลงหน่วยต้องมี กิโลเมตร เป็นอย่างน้อย
อายุแถวสูงสุดก่อนถูกตัดทิ้ง	12 เดือนนับจากวันที่สร้างแถว	`FgiConstant.INTERVAL_MONTH = 12`
ค่าแทน "ไม่มีวันยกเลิก"	`4000-01-01` (`to_date('01/4000','mm/yyyy')`)	ใช้กับ `COALESCE(cancel_date, ...)` ทั้ง c1 และ c2

ข. ชุด "คู่ร้านที่มีอยู่แล้ว" — ตัวตัดสินว่าเป็นคู่ใหม่หรือไม่

ชุดนี้ ไม่ใช่แค่ "แถวของงวดนี้" — ร้าน I ที่ยังมีรอบชดเชย active (`flag_action IN ('Y','W')`) ครอบคลุมงวดที่ขอ ก็นับว่า "มีอยู่แล้ว" ทั้งที่คู่ (I,N) นั้นอาจมาจากงวดอื่น · จุดนี้คือสาเหตุที่คู่ร้านบางคู่หายไปเงียบ ๆ ถ้า implement เป็นแค่ `WHERE impact_month = :m`

```
-- bind ตามลำดับ: $1=impact_month
-- ชุด "มีอยู่แล้ว" ของงวด :impact_month ('YYYY-MM')
-- แปลงตรงจาก ImportStoreJdbc.getImpactStoreFranchise (บรรทัด 170-219)
SELECT DISTINCT fis.impacted_store_code, fis.new_store_code
FROM sgi_fgi_impact_stores fis
LEFT JOIN sgi_fgi_impact_processes op
      ON op.flag_action IN ('Y', 'W') -- นับเฉพาะรอบที่ยัง active
     AND op.impacted_store_code = fis.impacted_store_code
     AND to_date($1 /* impact_month */, 'YYYY-MM') BETWEEN
           to_date(op.start_compensate_month, 'YYYY-MM')
         AND CASE
             -- ถ้า "เดือนก่อนหน้าเดือนปัจจุบัน" = งวดปิดล่าสุด หรือ งวดปิดล่าสุด + 1 เดือน
             -- ให้ยืดขอบบนออกไปถึงเดือนก่อนหน้า (รองรับรอบที่ยังชดเชยต่อเนื่องอยู่)
             WHEN (date_trunc('month', CURRENT_DATE) - INTERVAL '1 month') IN (
                   to_date(op.end_compensate_month, 'YYYY-MM'),
                   to_date(op.end_compensate_month, 'YYYY-MM') + INTERVAL '1 month')
             THEN date_trunc('month', CURRENT_DATE) - INTERVAL '1 month'
             ELSE to_date(op.end_compensate_month, 'YYYY-MM')
             END
WHERE fis.verify_status <> 'N' -- แถวที่ถูกตัดทิ้งไม่นับว่ามีอยู่
  AND ( fis.impact_month = $1 /* impact_month */ -- แถวของงวดนี้
       OR op.id IS NOT NULL ); -- หรือร้าน I อยู่ในรอบ active ที่คลุมงวดนี้

-- candidate จาก ALLMAP ที่ (impacted_store_code, new_store_code) ไม่อยู่ในผลลัพธ์นี้ = คู่ใหม่ -> INSERT
-- ที่อยู่ในผลลัพธ์ = ข้าม (นับเป็น skipped) ห้าม UPDATE ทับ
```

ค. กฎตัดทิ้ง และกฎเข้ากระบวนการ — **ต้องรันตามลำดับนี้เท่านั้น** (ตัดทิ้งก่อนเสมอ)

```
-- ● อ่านสองข้อนี้ก่อนใช้ SQL ด้านล่าง — ทั้งคู่ยังไม่ตรงกับระบบเดิม 100%
--
-- (1) แหล่งของ branchtype — SQL นี้ใช้ ms_i.status_type / ms_n.status_type จาก mas_store
-- แต่ระบบเดิมใช้ decode(fis.create_by,'STA', ms_i.status_type, fis.branchtype_i)
-- (ImportStoreJdbc.java:229) => แถว 'ALM' ซึ่งเป็นแถวที่ Job 2 สร้าง **ใช้ค่าจาก ALLMAP ไม่ใช่ master**
-- การเปลี่ยนมา join master เป็นการ **เปลี่ยนพฤติกรรมทางธุรกิจ** (ผล P/N อาจต่างจากเดิม)
-- ไม่ใช่แค่ปรับโครงสร้าง — ช่องว่าง G3 จึงยังไม่ปิด ต้องเคาะก่อน (DECISIONS ข้อ 2.15)
--
-- (2) การเลือกสัญญา fr_store — LEFT JOIN ตรง ๆ ข้างล่างยัง **ไม่รักษากติกาเดิม**
-- ร้านหนึ่งร้านมีหลายสัญญาในงวดเดียวได้ ระบบเดิมจึงคัดเหลือฉบับเดียวก่อนเสมอ:
--
-- dense_rank() OVER (PARTITION BY store_id, งวด
-- ORDER BY start_date DESC, cancel_type, order_id DESC) = 1
-- WHERE COALESCE(status,'-') <> 'D'
-- AND store_id <> '00000'
-- AND cancel_type IS NOT NULL
-- AND TRUNC(start_date) <= วันสุดท้ายของงวด
-- AND COALESCE(cancel_date, DATE '4000-01-01') >= วันแรกของงวด
--
-- ถ้าไม่ใส่ชั้นนี้ JOIN จะได้หลายแถวต่อร้าน => นิติบุคคล/วันสัญญาที่เอามาตัดสินอาจเป็นคนละฉบับ
-- กับที่ระบบเดิมใช้ และ UPDATE จะได้ผล P/N ไม่คงที่ **ต้องทำเป็น CTE คัดให้เหลือ rank 1 ก่อน join**

-- ค1. ตัดทิ้ง : รันก่อน (updateImpactStoreByJuristicNotMeetCondition · บรรทัด 344-380)
UPDATE sgi_fgi_impact_stores a
   SET verify_status = 'N', updated_by = 'ALM', updated_at = CURRENT_TIMESTAMP
  FROM mas_store ms_i -- ร้านที่ถูกกระทบ
  LEFT JOIN fr_store s_i ON s_i.store_id = ms_i.branch_id AND COALESCE(s_i.status,'-') <> 'D'
  LEFT JOIN juristic j_i ON j_i.juristic_id = s_i.juristic_id
     , mas_store ms_n -- ร้านเปิดใหม่
  LEFT JOIN fr_store s_n ON s_n.store_id = ms_n.branch_id AND COALESCE(s_n.status,'-') <> 'D'
  LEFT JOIN juristic j_n ON j_n.juristic_id = s_n.juristic_id
 WHERE a.verify_status = 'W'
   AND ms_i.branch_id = a.impacted_store_code
   AND ms_n.branch_id = a.new_store_code
   AND 'ALM' IN (a.created_by, a.updated_by)
   AND (
        ms_i.status_type NOT IN ('B','FAM','FB1','FB2','FC1','FVB','FVC','FPT1')
     OR ms_n.status_type IN ('F')
     OR TRIM(COALESCE(j_n.juristic_name,'juristic_n')) = TRIM(COALESCE(j_i.juristic_name,'juristic_i'))
     OR ( ( ms_i.status_type IN ('B','FAM','FB1','FB2','FC1','FVB','FVC')
            OR (ms_i.status_type = 'FPT1' AND s_i.cancel_type = '06') )
          AND ( date_trunc('day', s_i.start_date)
                >= (to_date(a.impact_month,'YYYY-MM') + INTERVAL '1 month - 1 day')
             OR to_date(a.impact_month,'YYYY-MM')
                >= date_trunc('day', COALESCE(s_i.cancel_date, DATE '4000-01-01')) ) )
     OR CURRENT_TIMESTAMP > (date_trunc('day', a.created_at) + INTERVAL '12 months')
   );
```

```
-- ค2. เข้ากระบวนการ : รันหลัง ค1 เสมอ (updateImpactStoreByJuristicMeetCondition · บรรทัด 381-420)
UPDATE sgi_fgi_impact_stores a
   SET verify_status = 'P', updated_by = 'ALM', updated_at = CURRENT_TIMESTAMP
 WHERE a.verify_status = 'W'
   AND (
       ( a.created_by = 'ALM'
        AND <ผ่านเกณฑ์ประเภทสาขาฝั่ง I> -- เงื่อนไขชุดเดียวกับ ค1 แต่กลับด้าน
        AND <ประเภทสาขาฝั่ง N ไม่ใช่ 'F'>
        AND <นิติบุคคลสองฝั่งคนละราย>
        AND <สัญญา SBP ของร้าน I คลุมงวด> )
     OR a.created_by = 'STA' -- STA ส่งเข้ามาเอง: ผ่านทันที ไม่ตรวจเกณฑ์ด้านบน
   );
```

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
ตัวกระตุ้น (Trigger)	S3 Object Created → EventBridge rule → AWS Batch SubmitJob	แก้ไขได้	มติ 2026-10-01 · rule กรองถึง prefix ชื่อไฟล์ interface/in/ALM/Impact_Store_Predict_ · ชุดตั้งค่าอยู่ใน output/aws/sgi-s3-triggers/ (สร้างด้วย tools/build_sgi_s3_triggers.py)
INPUT (JSON)	{"s3Key":"interface/in/ALM/Impact_Store_Predict_20260929.txt"}	แก้ไขได้	event ส่ง s3Key มาเอง → ทำไฟล์นั้นไฟล์เดียว · ไม่ส่ง s3Key = หยิบไฟล์ล่าสุด (รันมือ/rerun) · ฟิลด์ที่รับ: year · month · zones · dryRun · limit · s3Key · ปีเป็น ค.ศ.
ไฟล์ต้นทาง (EAI S3)	interface/in/ALM/Impact_Store_Predict_YYYYMMDD.txt	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	● เปลี่ยนต้นทาง 2026-09-29 จากวิว allmapssa.SEVEN_IMPACT_VIEW (SQL Server) · pipe-delimited · UTF-8 · LF · มีหัวตาราง 21 คอลัมน์ · กรองงวด + เลือกแถวแรกต่อคู่ร้าน (DISTANCE ASC, STORECODE_N ASC) ทำในโค้ด
ย้ายไฟล์เข้า backup	ย้ายเสมอ (ไม่มีสวิตช์ env · มติ 2026-10-15)	แก้ไขได้	ย้ายไป interface/backup-in/ALM/ เฉพาะรอบที่ไม่ล้มเหลวและไม่ใช่ dryRun
Branch Type ที่เข้าเกณฑ์	B, FAM, FB1, FB2, FC1, FVB, FVC, FPT1	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	FPT1 เข้าเกณฑ์เฉพาะเมื่อ SBP_CANCEL_TYPE_I = 06
กฎ DENY (ตรวจก่อน ON_PROCESS)	สาขา N=F / juristic เดียวกัน / สัญญาไม่คลุมงวด / เก่ากว่า 12 เดือน	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	
PK Sequence	BIGSERIAL ของ sgi_fgi_impact_stores (PostgreSQL — ไม่ใช้ named sequence แบบ Oracle)	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	

2.6 หลักฐานที่ต้องเก็บตอนรัน

Evidence	Job-specific value	Acceptance
Input identity	Period year/month, optional zone filter, and ALLMAP Impact_Store_Predict_*.txt rows read from EAI S3 (interface/in/ALM/).	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	FGI_IMPACT_STORE and related impact/new-store tables contain imported candidates for the requested period with duplicate-safe status.	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	UNIQUE(impacted_store_code, new_store_code, impact_month) + `ON CONFLICT DO NOTHING`; คู่ร้านที่มีอยู่แล้วต้องข้ามเงียบและนับเป็น `skipped` ห้ามอัปเดตทับ (พฤติกรรมเดิมของระบบ)	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ

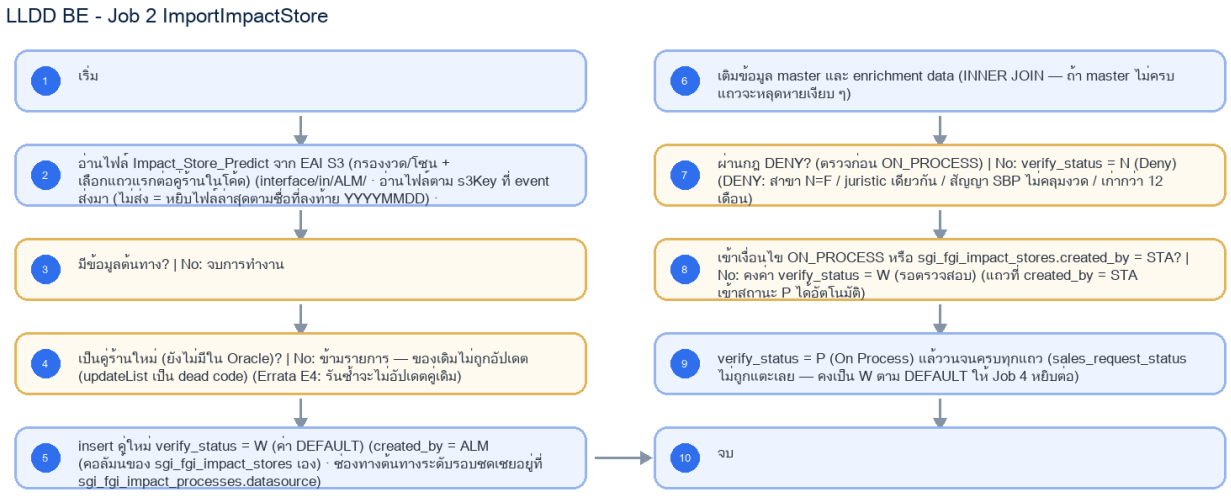
Evidence	Job-specific value	Acceptance
Transaction proof	ทั้ง 3 ขั้น (impacted_stores -> impact_processes -> impact_stores) ของ candidate หนึ่งราย ต้องอยู่ใน transaction เดียวกัน — ห้าม commit แถวแม่แยกจากคู่ร้านไม่งั้นจะเหลือแถวแม่ที่ไม่มีลูก; chunk fail rollback เฉพาะ chunk แล้วนับเข้า failedCount; ห้ามซ้อน transaction สองชั้นแบบระบบเดิม	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	อ่านไฟล์จาก EAI S3 ด้วย IAM role ของ AWS Batch (ไม่มี credential ในโค้ดหรือพารามิเตอร์) · bucket/prefix มาจาก env S3_BUCKET_ALM/S3_KEY_ALM · ย้ายไฟล์เข้า backup เสมอเมื่อนำเข้าสำเร็จ (มติ 2026-10-15 · ไม่มีสวิตช์ env) และย้ายเฉพาะรอบที่ไม่ล้มเหลว	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

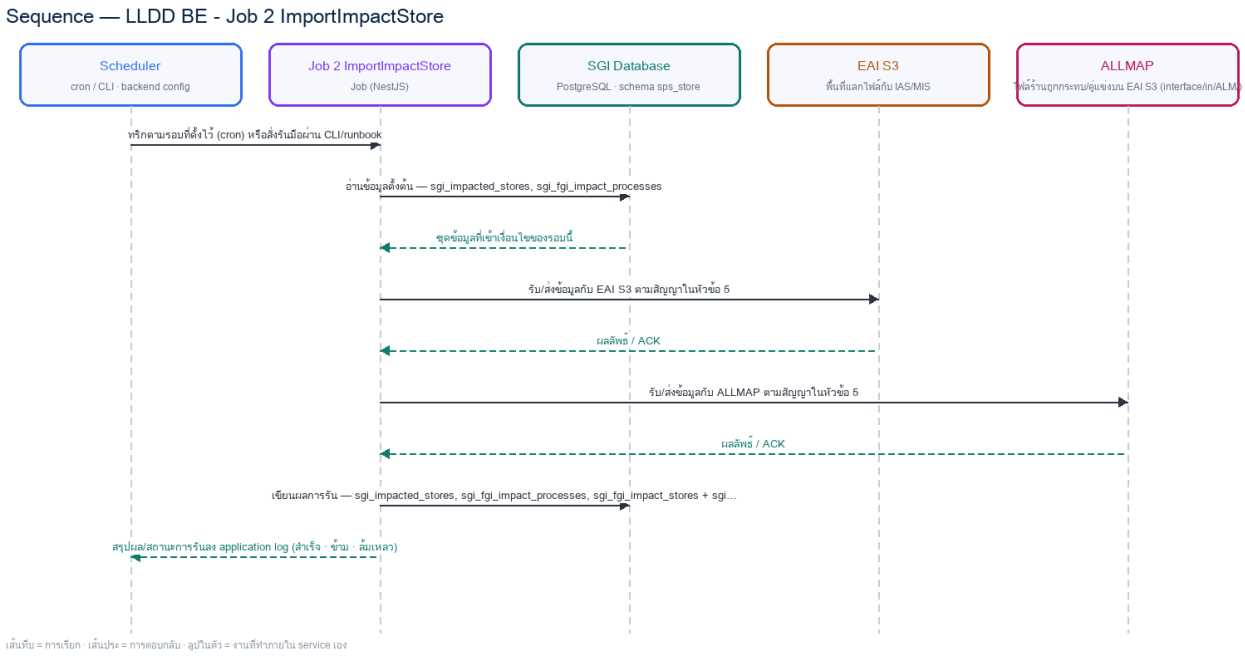
Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ImportImpactStore.java	24-186	Legacy main entrypoint for impacted-store import.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java	30-84, 170-484	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 2 ImportImpactStore



รูปที่ 2: Sequence diagram: LLDD BE - Job 2 ImportImpactStore

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sgi_impacted_stores	R/W	upsert ร้านที่ถูกกระทบก่อน (FK ต้นทาง) — ห้ามเขียนทับค่าที่ Job 2 ไม่มีข้อมูล เช่น opt_dv_user_id / latitude / longitude
sgi_fgi_impact_processes	R/W	upsert แถวแม่ของ (ร้าน + งวด) แล้วอ่าน id กลับมาใส่ impact_process_id — ON CONFLICT DO NOTHING ไม่คืนแถว ต้อง SELECT ซ้ำ
sgi_fgi_impact_stores	W	insert คู่ร้านกระทบ–ร้านใหม่ (verify_status = W ตาม DEFAULT) / ตั้ง verify_status เป็น P หรือ N · sales_request_status คงเป็น W ให้ Job 4 หยิบต่อ

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-2` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-2-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-2-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-02-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (`BATCH_START`/`BATCH_END` + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

53 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

Argument / Input Validation

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	แปลง raw argument รูปแบบเดิม (`ZONE|YEAR|MONTH`) เป็นรูปแบบใหม่ (named option) — ทดสอบ 2 กรณีย่อย: (1) ระบุ zone ตรงๆ (2) zone เป็น `ALL`	(1) `BN,BS|2026|07` (2) `ALL|2026|07`	(1) ได้ `zones=BN,BS, year=2026, month=07` (2) ได้ `zones=ALL, year=2026, month=07`
UT-02	Normalize zone ที่มีเว้นวรรค/ตัวพิมพ์เล็ก	`zones=' bn , bs '`	ได้ `['BN', 'BS']` (trim + ตัวพิมพ์ใหญ่)
UT-03	เติม default งวดก่อนหน้าเมื่อไม่ระบุปี/เดือน (รวมกรณีข้ามปี)	ไม่ระบุ `year/month`; ทดสอบเวลาปัจจุบัน = 15 ส.ค. 2026 และ 10 ม.ค. 2026 (Asia/Bangkok)	ได้ `year=2026,month=7` และ `year=2025,month=12` ตามลำดับ
UT-04	validate ไม่ผ่านเมื่อระบุ `year/month` ไม่ครบคู่ หรือ `month` นอกช่วง 1-12	เช่น `year=2026` ไม่ระบุ `month`, หรือ `month=13`	validate ไม่ผ่าน → error, ไม่เรียก Service
UT-05	ไม่ filter zone เมื่อไม่ระบุ `zones` หรือระบุ `ALL`	`zones=ALL` หรือไม่ระบุ `zones` เลย	validate ผ่าน ไม่ filter zone ตอนประมวลผล
UT-06	ไม่ระบุ argument ใดๆเลยทั้งหมด	raw argument ว่าง (`[]`)	validate ผ่านตามปกติ (ไม่ใช่ error): ไม่ filter zone และเติม default งวดเป็นเดือนก่อนหน้า แล้วเรียก Service ต่อได้

การดึงไฟล์จาก S3

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-07	มีหลายไฟล์ วันที่ในชื่อไฟล์ต่างกัน	ไฟล์ `..._20260709.txt`, `..._20260701.txt`	เลือกไฟล์ `20260709`
UT-08	วันที่ในชื่อไฟล์เท่ากัน แต่เวลาแก้ไขต่างกัน	ไฟล์ 2 ตัว `YYYYMMDD` เดียวกัน, `LastModified` ต่างกัน	เลือกไฟล์ที่ `LastModified` ใหม่ล่าสุด
UT-09	ต้อง list ไฟล์หลายหน้า	S3 คืนผลแบบแบ่งหน้า 2 หน้า	รวมไฟล์จากทุกหน้าก่อนเลือกไฟล์ล่าสุด
UT-10	ไม่มีไฟล์ตรง pattern เลย	ไฟล์ในถังไม่มีชื่อที่ตรง `Impact_Store_Predict_YYYYMMDD.txt`	สรุปผลเป็น `FAILED` ด้วย error ไม่พบไฟล์
UT-11	เรียก S3 แล้ว error	S3 ตอบ error ตอน list	สรุปผลเป็น `FAILED`

การแปลงข้อมูล (Parsing)

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-12	header ไม่ถูกต้อง	ไฟล์ที่บรรทัดแรกผิด format	สรุปผลเป็น `FAILED`, ไม่อ่าน row ข้อมูลต่อ
UT-13	stream ขาดกลางทาง	stream ที่หยุดกลางไฟล์	สรุปผลเป็น `FAILED`, ไม่คืนผลลัพธ์แบบครึ่งๆกลางๆ
UT-14	row มีตัวเลขผิดรูปแบบ	row ที่ค่าระยะทางเป็นตัวอักษร	row นั้นถูกเก็บเป็น rejected พร้อมเหตุผล, ไม่แปลงเป็น `0`, อ่าน row ถัดไปต่อ
UT-15	row มีวันที่ผิดรูปแบบ	row ที่วันเปิดร้านใหม่ผิด format	row นั้นถูก reject พร้อมเหตุผล, ไม่แปลงเป็นค่าว่าง
UT-16	row ถูกต้องครบ	row ที่ทุก field ถูกต้อง	เก็บเข้า accepted list พร้อมสถานะ `W` และ `create_by=ALM`
UT-17	มีเครื่องหมาย delimiter อยู่ใน double quote	row ที่มี `"a|b"` ในคอลัมน์	ไม่ตัด column ผิดตำแหน่ง

การย้ายไฟล์ไป Backup

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-18	copy และลบสำเร็จทั้งคู่	mock copy/delete สำเร็จ	เรียก copy ก่อน แล้วลบไฟล์ต้นทางตามลำดับ
UT-19	copy ล้มเหลว	mock copy ล้มเหลว	ไม่เรียกลบไฟล์เลย, สรุปผลเป็น `FAILED`, ไฟล์ต้นทางยังอยู่
UT-20	copy สำเร็จ แต่ลบล้มเหลว	mock copy สำเร็จ, ลบล้มเหลว	สรุปผลเป็น `FAILED`, ไฟล์ backup ที่ copy ไปแล้วยังอยู่, log ว่าต้นทางยังอยู่

Duplicate / เงื่อนไขข้าม

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-21	zone filter ไม่ตรง เหลือ record เป็นศูนย์	filter zone ที่ไม่มีร้านตรง	สรุปผลเป็น `NO_DATA` ไม่เปิด transaction
UT-22	record ไม่ซ้ำกับข้อมูลในฐานข้อมูล	key ไม่ตรงกับผลจากการ query	เพิ่มเข้า list รายการใหม่
UT-23	record ซ้ำกับข้อมูลที่มีอยู่แล้ว	key ตรงกับผลจากการ query	ข้าม record นั้น ไม่เพิ่มเข้า list
UT-24	record ซ้ำกันเองภายในไฟล์เดียวกัน	2 record key เดียวกันในไฟล์เดียวกัน	เก็บ record แรก, ข้าม record ที่ซ้ำถัดมา
UT-25	คู่ร้านย้อนหลังอยู่ใน active/closing process window ของงวดเป้าหมาย	`storecode_i` ตรงกับ `sgi_fgi_impact_processes` ที่ `flag_action IN ('Y','W')` และงวดเป้าหมายอยู่ระหว่าง `start_compensate` ถึง `end_period` แบบ inclusive (ไม่มี record ตรงงวดใน `sgi_fgi_impact_stores` เลย)	ถือว่าซ้ำ, ถูกกันออกจาก `newImpactStoreList`
UT-26	คู่ร้านย้อนหลังอยู่นอก active/closing process window	งวดเป้าหมายอยู่นอกช่วง `start_compensate..end_period`	ไม่ถือว่าซ้ำ, record ผ่านเข้า `newImpactStoreList` ได้ตามปกติ
UT-27	ไม่มีรายการใหม่เลยหลังกันซ้ำ	list รายการใหม่ว่าง	สรุปผลเป็น `SUCCESS` ไม่เปิด transaction

การบันทึกข้อมูล / Transaction / เงื่อนไข

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-28	insert รายการใหม่สำเร็จ	record ที่ยังไม่มีคู่ซ้ำ	ได้ id ของแถวที่ insert ครบทุก record
UT-29	insert ชนกับ record ที่สถานะยังไม่ใช่ N	record key ซ้ำกับแถวที่ `flag_verify <> 'N'`	ไม่ insert แถวนั้น ไม่ถือเป็น error
UT-30	re-import คู่ที่มีแต่แถวสถานะ N	seed แถวเดิมเป็น `N` แล้ว insert คู่เดิมอีกครั้ง	insert สำเร็จ (ไม่ชน unique index)
UT-31	update ข้อมูลร้านเปิด/นิติบุคคลจำกัดด้วย id	id เฉพาะรอบนี้ (`flag_verify='W'`)	เฉพาะแถวที่ id ตรงถูก update, แถวอื่นไม่กระทบ
UT-32	update ข้อมูลร้านเปิด/ร้านเดิม: record มาจาก STA เทียบกับไม่ได้มาจาก STA	`create_by='STA'` เทียบกับ `create_by='ALM'`	STA ใช้ข้อมูล branch name/open date/region/status type จาก `mas_store` แทนค่า import; `ALM` คงค่าที่ import มาจากไฟล์
UT-33	W→N: ประเภทสาขาร้านเดิมไม่อยู่ในกลุ่มที่ติดตาม	`branchtype_i` ไม่อยู่ในกลุ่มที่กำหนด	เปลี่ยนเป็น `N`
UT-34	W→N: ร้านใหม่เป็นประเภท F	`branchtype_n='F'`	เปลี่ยนเป็น `N`
UT-35	W→N: นิติบุคคลเหมือนกัน	ชื่อนิติบุคคลร้านเดิม=ร้านใหม่ (หลัง trim)	เปลี่ยนเป็น `N`
UT-36	W→N: SBP ไม่ active ครอบงวด	วันเริ่ม SBP ตรงหรือหลังวันสุดท้ายของงวด	เปลี่ยนเป็น `N`
UT-37	W→N: ค้างสถานะ W เกิน 12 เดือน	`create_date` เก่ากว่า 12 เดือน	เปลี่ยนเป็น `N`
UT-38	W→N: record ไม่เกี่ยวกับ `ALM` เลย แม้เข้าเงื่อนไข deny ข้ออื่นครบ	`create_by` และ `update_by` ทั้งคู่ไม่ใช่ `ALM` (เช่นเป็น STA ทั้งคู่) แต่ `branchtype_n='F'`	ไม่ ถูกเปลี่ยนเป็น `N` (ไม่ผ่านเงื่อนไข ALM guard)
UT-39	W→N: ไม่เข้าเงื่อนไขใดเลย	record ที่ผ่านทุกเกณฑ์	ยังคงเป็น `W` (ไม่ถูกแก้ไข)
UT-40	W→P: สร้างโดย ALM และเข้าเงื่อนไขครบ	ตามเงื่อนไข W→P ทุกข้อ	เปลี่ยนเป็น `P`
UT-41	W→P: สร้างโดยช่องทางอื่น (STA)	`create_by='STA'`	เปลี่ยนเป็น `P` โดยไม่ต้องเช็คเงื่อนไข ALM
UT-42	W→P: ไม่เข้าเงื่อนไขทั้ง N และ P	เช่น `FPT1` ที่ยกเลิกด้วยประเภทอื่นที่ไม่ใช่ `06`	ยังคงเป็น `W` และถูกนับใน `remaining_w_rows`
UT-43	ลำดับ N ต้องมาก่อน P เสมอ	record ที่เข้าเงื่อนไข N ด้วย	ถูกเปลี่ยนเป็น `N` และไม่ถูก promote เป็น `P` ในขั้นตอนถัดมา
UT-44	เกิด error ฐานข้อมูลระหว่าง transaction	mock ขั้นตอนใดขั้นตอนหนึ่ง (insert/update) ให้ error	rollback ทั้งหมด, ไม่มีข้อมูลค้างในฐานข้อมูล, สรุปผลเป็น `FAILED`

กรณีไม่มีข้อมูล (No Data)

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-45	ทุก row ในไฟล์ผิดรูปแบบทั้งหมด	ทุก row ถูก reject	สรุปผลเป็น `NO_DATA`, `rejected_rows = total_rows`, `accepted_rows = 0`
UT-46	มี accepted record แต่ซ้ำกับข้อมูลเดิมทั้งหมด	ทุก record ถูกกันซ้ำหมด	สรุปผลเป็น SUCCESS พร้อม `inserted_rows = 0`

ผลลัพธ์มาตรฐานและ Error

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-47	เกิด error กลางทาง (เช่น database error ระหว่าง transaction)	mock error หลังผ่านขั้นตอน S3/parse บางส่วนไปแล้ว	ผลลัพธ์ `FAILED` ยังคง `year`, `month`, `source_file`, `started_at`/`finished_at` และ counters ที่ทราบก่อนเกิด error ไว้ครบ
UT-48	error message ที่แสดงใน result ต้อง sanitize	error ที่มี connection string/credential อยู่ใน stack trace เดิม	`error.message`/`error.cause` ไม่มีข้อมูลอ่อนไหว (เช่น connection string, credential) หลุดออกมา

การแจ้งเตือน / Notification

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-49	หา template และส่งอีเมลสำเร็จ	mock ทั้งสองขั้นตอนสำเร็จ	อีเมลถูกส่งด้วยเนื้อความที่ render จากผลลัพธ์จริง
UT-50	หา template ไม่พบ	mock query template คืนค่าว่าง	log ไว้เท่านั้น ไม่ throw error, ไม่กระทบผลลัพธ์ที่ commit แล้ว
UT-51	ส่งอีเมลผ่าน SMTP ล้มเหลว	mock SMTP ล้มเหลว	log ไว้เท่านั้น, ผลลัพธ์ `SUCCESS`/`NO_DATA` เดิมไม่เปลี่ยนเป็น `FAILED`
UT-52	ผลลัพธ์เป็น `SUCCESS`/`NO_DATA`	ทำงานจบปกติ	exit code = `0`
UT-53	ผลลัพธ์เป็น `FAILED`	เกิด error ณ จุดใดก็ตาม	exit code = `1`

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ImportImpactStore.java — lines 24-35

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ImportImpactStore.java
Original lines	24-35
Responsibility	Legacy main entrypoint for impacted-store import.

```
public class ImportImpactStore {

private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");
private static FgiConfig config = (FgiConfig) context.getBean("fgiConfig");
private static ImpactStoreService storeImpactService = (ImpactStoreService) context.getBean("storeImpactService");
    public static Locale LOCALE_EN = Locale.US;
    public static String DATE_FORMAT_DDMMYYYY_TIME = "dd/MM/yyyy HH:mm:ss";

public static void main(String[] args) {
LogUtils.info(ImportImpactStore.class,"============****** Start => import FGI_IMPACT_STORE ******============");
Calendar cal = Calendar.getInstance(Locale.US);
String year = th.co.gosoft.fcs.utils.DateUtils.modifyDateToString( new Date() , -1 , "yyyy" , LOCALE_EN ) ;
```

J1. ImportImpactStore.java — lines 175-186

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ImportImpactStore.java
Original lines	175-186
Responsibility	Legacy main entrypoint for impacted-store import.

```
email.send();
LogUtils.info(ImportImpactStore.class,"---- send email : [success] ----");
statusSend = true ;
}catch(Exception e){
statusSend = false ;
LogUtils.error(ImportImpactStore.class,e);
LogUtils.info(ImportImpactStore.class,"---- send email : [fail] ----");
}
return statusSend ;
}

}
```

J2. ImportStoreJdbc.java — lines 30-41

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	30-41
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```
public List<ImpactStore> selectVsevenImpact(int year, int month,String[] zone) {
String zoneStr = new String();
try{
List params = new LinkedList();
String sql = " SELECT * ";
sql += " FROM ( ";
sql += " SELECT *, ";
sql += " ROW_NUMBER() OVER(PARTITION BY STORECODE_I, STORECODE_N, PERIOD_YEAR, PERIOD_MONTH ORDER BY PERIOD_YEAR,
PERIOD_MONTH) AS ROW_NUM ";
sql += " FROM " +FgiConstant.ALLMAP_SCHEMA_PRD +".SEVEN_IMPACT_VIEW ";
sql += " WHERE PERIOD_MONTH = ? AND PERIOD_YEAR = ? ";
if(zone != null && zone.length >0){
zoneStr = TextUtils.arrayElementToStringHasQuote(zone);
```

J2. ImportStoreJdbc.java — lines 73-84

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	73-84
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```
sql.append(" SEQ_FGI_IMPACT_STORE.NEXTVAL, ?, ?, ");
sql.append(" ?, ?, ?, ?, ?, ?, ");
sql.append(" ?, ?, ?, ?, ?, ?, ");
sql.append(" ?, ?, ?, ?, ?, ");
sql.append(" ? , ? ");
sql.append(" ) ");
int[] result = jdbcTemplate.batchUpdate(sql.toString(), new ImportImpactStoreBatch(impactStoreList));
status = true;
LogUtils.info(getClass(),"----- insert FGI_IMPACT_STORE : " + result.length + " -----");

return status;
}
```

J2. ImportStoreJdbc.java — lines 170-181

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	170-181
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```
public List<ImpactStore> getImpactStoreFranchise(int year, int month,String[] zoneArr) {
StringBuffer sql = new StringBuffer();
String zoneStr = new String();
List params = new LinkedList();
try{
sql.append(" select distinct fis.storecode_i, fis.storecode_n, ? as month, ? as year ");
sql.append(" from fgi_impact_store fis ");
sql.append(" left join fgi_impact_store_on_process op on op.flag_action in ('Y','W') ");
sql.append(" and op.storecode_i = fis.storecode_i ");
sql.append(" and to_date(? || ?, 'YYYYMM') ");
sql.append(" between to_date(op.start_compensate_year || op.start_compensate_month, 'YYYYMM') ");
sql.append(" and ( ");
```

J2. ImportStoreJdbc.java — lines 473-484

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ImportStoreJdbc.java
Original lines	473-484
Responsibility	Query SEVEN_IMPACT_VIEW and insert/update FGI impact/new-store records.

```
}catch(Exception e){
LogUtils.error(getClass(),e);
return jdbcTemplate.batchUpdate(sql.toString(), new UpdateImpactSaleFromIASBatch(impactStoreList));
}
}

public EmailTemplateBean getEmailTemplateByPk(String emailTemplateId) {

StringBuffer sql = new StringBuffer();
sql.append("SELECT SUBJECT_FORMAT, BODY_FORMAT ,SENDER FROM EMAIL_TEMPLATE ");

if(StringUtils.isNotEmpty(emailTemplateId)){
```